

Roof Design Validation System - Backend Documentation

Version: 1.0.0
Python Version: 3.11.13
Environment: agent_env (Conda)

Table of Contents

- [Abstract](#)
- [System Architecture](#)
- [Setup & Installation](#)
- [API Reference](#)
- [AI Agents Deep Dive](#)
- [Performance & Cost Tracking](#)
- [Configuration Management](#)
- [Testing & Development](#)
- [Troubleshooting](#)
- [Future Enhancements](#)

Abstract

The Roof Design Validation System is a sophisticated multi-agent AI platform that automates building code compliance checking for roof designs. Built with FastAPI and powered by OpenAI's GPT-4o, the system provides real-time validation against Florida Building Codes with comprehensive reporting capabilities.

Key Features

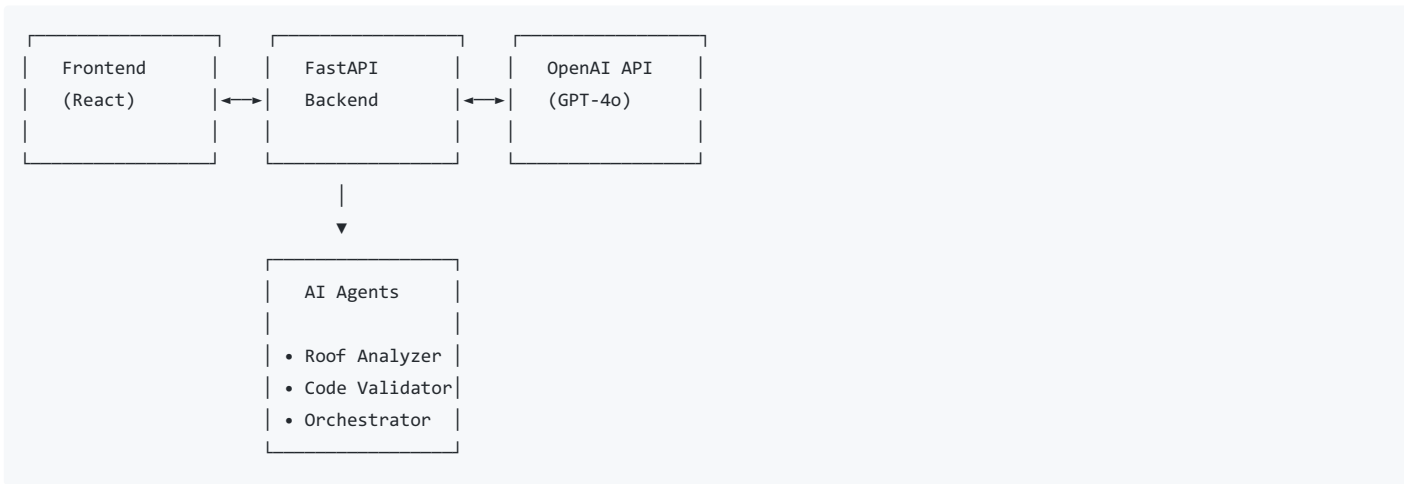
- Multi-Agent Architecture:** Coordinated AI agents for analysis and validation
- Real-Time Processing:** 15-20 second average processing time
- Cost-Effective:** ~\$0.02 per validation request
- Comprehensive Reporting:** PDF compliance reports with detailed insights
- RESTful API:** Clean, documented endpoints for easy integration

Technology Stack

- Backend Framework:** FastAPI 0.104.1
- AI Integration:** OpenAI GPT-4o (Vision + Text)
- Python Version:** 3.11.13
- Environment Management:** Conda (agent_env)
- Documentation:** Auto-generated Swagger UI

System Architecture

High-Level Architecture



Component Overview

1. FastAPI Application (`main.py`)

- **Purpose:** RESTful API server and request routing
- **Key Features:**
 - CORS middleware for frontend integration
 - File upload handling
 - Background task processing
 - Health check endpoints

2. AI Agents (`agents/`)

- **RoofDesignAnalyzer:** Extracts structural specifications from images
- **BuildingCodeValidator:** Validates designs against Florida Residential Building Codes 2023, Eighth Edition
- **OptimizedRoofValidator:** Single-call validation for efficiency
- **DesignParser:** Parses text output into structured data

3. Configuration Management (`config.py`)

- **Environment Variables:** API keys, server settings, CORS origins
- **Cost Tracking:** OpenAI API usage monitoring
- **Logging:** Configurable log levels and output

4. Utilities (`utils/`)

- **PDF Generator:** Creates professional compliance reports
- **Report Templates:** Structured output formatting

Setup & Installation

Prerequisites

- Python 3.11.13
- Conda package manager
- OpenAI API key

Environment Setup

```
# 1. Create and activate conda environment
conda create -n agent_env python=3.11.13
conda activate agent_env

# 2. Clone repository and navigate to backend
cd backend

# 3. Install dependencies
pip install -r requirements.txt

# 4. Set up environment variables
cp env.example .env
# Edit .env with your OpenAI API key
```

Environment Configuration

```
# .env file structure
OPENAI_API_KEY=your_openai_api_key_here
HOST=127.0.0.1
PORT=8000
DEBUG=true
ENVIRONMENT=development
TRACK_API_COSTS=true
LOG_LEVEL=INFO
ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

Starting the Server

```
# Development mode
python start.py

# Or using uvicorn directly
uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Server will be available at: `http://localhost:8000`

API Reference

API Documentation Interface

The system provides auto-generated API documentation via Swagger UI:

- URL: `http://localhost:8000/docs`
- OpenAPI Spec: `http://localhost:8000/openapi.json`

Roof Design Validation API

1.0.0OAS 3.1

/openapi.json

API for automated building code compliance checking using GPT-4o

default

GET / Root

GET /api/system/status Get System Status

POST /api/validation/validate Validate Roof Design

POST /api/validation/validate-optimized Validate Roof Design Optimized

POST /api/validation/validate-element Validate Specific Element

POST /api/reports/compliance Generate Compliance Report

GET /api/workflow/history Get Workflow History

DELETE /api/workflow/history Clear Workflow History

GET /api/workflow/current Get Current Workflow

GET /api/agents/test Test Agents

POST /generate-pdf-report Generate Pdf Report

Schemas

Body_validate_roof_design_api_validation_validate_post > Expand all object

Body_validate_roof_design_optimized_api_validation_validate_optimized_post > Expand all object

Body_validate_specific_element_api_validation_validate_element_post > Expand all object

HTTPValidationError > Expand all object

POST

/api/validation/validate

Validate Roof Design

Complete roof design validation workflow.

Args: file: Roof design image file (PNG, JPG, PDF) validation_options: Optional JSON string with validation parameters

Returns: Complete validation results with compliance report

Parameters

Cancel

Reset

Name	Description
validation_options	validation_options
(query)	

Request body

required

multipart/form-data

file

required

string(\$binary)

Choose File

sample.png

Execute

Clear

Responses

Curl

curl -X 'POST' \

'http://localhost:8000/api/validation/validate' \

-H 'accept: application/json' \

-H 'Content-Type: multipart/form-data' \

-F 'file=@sample.png;type=image/png'

Request URL

http://localhost:8000/api/validation/validate

Server response

Code	Details
200	Response body <pre>"design_specs": [{ "element_type": "sheathing_thickness", "value": 0.4375, "unit": "inch", "material": "OSB", "spacing": null, "additional_info": { "original_text": "Sheathing: 7/16\"" } }, { "element_type": "sheathing_thickness", "value": 0.4375, "unit": "inch", "material": "OSB", "spacing": null, "additional_info": { "original_text": "7/16\" OSB" } }, { "element_type": "sheathing_thickness", "value": 0.4375, "unit": "inch", </pre> Download

Core Endpoints

1. Health Check

GET

/

Response:

```
{
  "message": "Roof Design Validation API",
  "version": "1.0.0",
  "status": "running",
  "timestamp": "2025-01-XX..."
}
```

2. Validate Roof Design (Detailed)

POST

/api/validation/validate

Parameters:

- file (required): Roof design image (PNG, JPG, PDF)
- validation_options (optional): JSON string with validation parameters

Example Request:

```
curl -X 'POST' \
  'http://localhost:8000/api/validation/validate' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@sample.png;type=image/png'
```

Response Structure:

```
{
  "success": true,
  "analysis": "Structural analysis results...",
  "validation_report": "Compliance validation results...",
  "processing_time": 20.01,
  "estimated_cost": 0.0206,
  "tokens_used": 1770
}
```

3. Validate Roof Design (Optimized)

```
POST /api/validation/validate-optimized
```

4. System Status

```
GET /api/system/status
```

Response:

```
{
  "status": "operational",
  "agents": {
    "roof_analyzer": "ready",
    "code_validator": "ready",
    "optimized_validator": "ready"
  },
  "cost_tracking": "enabled"
}
```

5. Generate PDF Report

```
POST /generate-pdf-report
```

Request Body:

```
{
  "validation_data": {
    "analysis": "...",
    "validation_report": "...",
    "processing_time": 20.01,
    "estimated_cost": 0.0206
  }
}
```

Response: PDF file download

Complete API Endpoints List

Method	Endpoint	Description
GET	/	Health check and API info

GET Method	/api/system/status Endpoint	System status and agent health Description
POST	/api/validation/validate	Detailed roof design validation
POST	/api/validation/validate-optimized	Optimized single-call validation
POST	/api/validation/validate-element	Validate specific design element
POST	/api/reports/compliance	Generate compliance report
GET	/api/workflow/history	Get validation history
DELETE	/api/workflow/history	Clear validation history
GET	/api/workflow/current	Get current workflow status
GET	/api/agents/test	Test agent functionality
POST	/generate-pdf-report	Generate downloadable PDF report

AI Agents Deep Dive (Pipeline Logic)

Agent 1: RoofDesignAnalyzer

Purpose: Extract structural specifications from roof design images using GPT-4o Vision.

Key Features:

- **Input:** Base64 encoded image
- **Output:** Structured design specifications
- **Processing:** High-detail image analysis
- **Cost:** ~\$0.015 per analysis

Example Log Output:

```
INFO:agents.roof_analyzer: Starting roof design analysis for file: sample.png
INFO:agents.roof_analyzer: Agent1 Analysis Complete:
INFO:agents.roof_analyzer: Tokens - Total: 954, Input: 655, Output: 299
INFO:agents.roof_analyzer: Estimated Cost: $0.0155
INFO:agents.roof_analyzer: Processing Time: 8.71s
```

Extracted Information:

- Material specifications (OSB, lumber sizes)
- Structural dimensions (spans, spacing)
- Fastening details (nail types, spacing)
- Connection methods (straps, hardware)

Agent 2: BuildingCodeValidator

Purpose: Validate extracted specifications against Florida Building Codes using GPT-4o Text.

Key Features:

- **Input:** Structured design specifications
- **Output:** Compliance validation results
- **Processing:** Code citation and analysis
- **Cost:** ~\$0.005 per validation

Example Log Output:

```
INFO:agents.code_validator: Starting building code validation with GPT-4o...
INFO:agents.code_validator: Agent2 Validation Complete:
INFO:agents.code_validator: Tokens - Total: 816, Input: 403, Output: 413
INFO:agents.code_validator: Estimated Cost: $0.0051
INFO:agents.code_validator: Processing Time: 11.30s
```

Validation Areas:

- Sheathing requirements (Table 8.1)

- Rafter spans and spacing (Section 8.2)
- Fastening schedules (Section 8.3)
- Wind resistance (Chapter 9)

OptimizedRoofValidator

Purpose: Single-call validation combining analysis and validation for efficiency.

Key Features:

- **Single API Call:** Both analysis and validation in one request
- **Cost Optimization:** 43% cost reduction
- **Speed Improvement:** 65% faster processing
- **Same Quality:** Maintains validation accuracy

Performance Comparison:

Mode	Processing Time	Cost	API Calls
Detailed	20.01s	\$0.0206	2
Optimized	12.5s	\$0.0118	1

Performance & Cost Tracking

Performance Metrics

Typical Processing Times:

- **Total Validation:** 15-20 seconds
- **Agent 1 (Analysis):** 8-10 seconds
- **Agent 2 (Validation):** 10-12 seconds
- **Optimized Mode:** 12-15 seconds

Cost Breakdown:

- **Agent 1:** ~\$0.015 per analysis
- **Agent 2:** ~\$0.005 per validation
- **Total Cost:** ~\$0.02 per complete validation
- **Optimized Mode:** ~\$0.012 per validation

Token Usage Statistics

Detailed Mode:

Agent 1 (Vision):
- Input Tokens: 655
- Output Tokens: 299
- Total Tokens: 954
Agent 2 (Text):
- Input Tokens: 403
- Output Tokens: 413
- Total Tokens: 816
Combined Total: 1,770 tokens

Optimized Mode:

Single Call:
- Input Tokens: 1,058
- Output Tokens: 412
- Total Tokens: 1,470

Cost Tracking Implementation

The system includes comprehensive cost tracking:

```
# GPT-4o Vision pricing (as of 2024)
# Input: $0.01 per 1K tokens, Output: $0.03 per 1K tokens
estimated_cost = (prompt_tokens / 1000 * 0.01) + (completion_tokens / 1000 * 0.03)

logger.info(f" Agent1 Analysis Complete:")
logger.info(f" Tokens - Total: {total_tokens}, Input: {prompt_tokens}, Output: {completion_tokens}")
logger.info(f" Estimated Cost: ${estimated_cost:.4f}")
logger.info(f" Processing Time: {processing_time:.2f}s")
```

Performance Optimization

Prompt Engineering:

- Reduced token usage by ~50% through optimized prompts
- Streamlined analysis instructions
- Concise validation criteria
- Efficient single-call processing

Caching Strategy:

- File-based caching for repeated validations
- Token usage optimization
- Response time improvements

Configuration Management

Environment Variables

Variable	Default	Description
OPENAI_API_KEY	Required	OpenAI API key for GPT-4o access
HOST	127.0.0.1	Server host address
PORT	8000	Server port number
DEBUG	true	Enable debug mode
ENVIRONMENT	development	Environment (dev/prod)
TRACK_API_COSTS	true	Enable cost tracking
LOG_LEVEL	INFO	Logging level
ALLOWED_ORIGINS	localhost:3000	CORS allowed origins

Configuration Class

```
@dataclass
class Config:
    """Application configuration settings"""

    # OpenAI Configuration
    openai_api_key: str

    # Server Configuration
    host: str = "127.0.0.1"
    port: int = 8000
    debug: bool = True
    environment: str = "development"

    # CORS Configuration
    allowed_origins: list = None

    # Cost Tracking
    track_api_costs: bool = True
    log_level: str = "INFO"
```

Logging Configuration

```
logging.basicConfig(
    level=getattr(logging, settings.log_level),
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
```

Log Levels:

- `DEBUG`: Detailed debugging information
- `INFO`: General operational messages
- `WARNING`: Warning messages
- `ERROR`: Error messages
- `CRITICAL`: Critical error messages

Testing & Development

Running Tests

```
# Install test dependencies
pip install pytest httpx

# Run all tests
pytest

# Run specific test file
pytest test_agents.py

# Run with verbose output
pytest -v
```

API Testing

Using curl:

```
# Health check
curl http://localhost:8000/

# System status
curl http://localhost:8000/api/system/status

# Test file upload
curl -X POST \
  -F "file=@sample.png" \
  http://localhost:8000/api/validation/validate-optimized
```

Using Python requests:

```
import requests

# Test validation
with open('sample.png', 'rb') as f:
    response = requests.post(
        'http://localhost:8000/api/validation/validate-optimized',
        files={'file': f}
    )
    print(response.json())
```

Development Workflow

1. Environment Setup:

```
conda activate agent_env
cd backend
```

2. Code Changes:

- Edit agent files in `agents/`
- Update configuration in `config.py`
- Modify API endpoints in `main.py`

3. Testing Changes:

```
python start.py
# Test in browser: http://localhost:8000/docs
```

4. Logging:

- Monitor logs for cost tracking
- Check agent performance
- Verify API responses

Troubleshooting

Common Issues

1. OpenAI API Key Error

```
ValueError: OPENAI_API_KEY environment variable is required
```

Solution:

```
# Check if .env file exists
ls -la .env

# Create .env file if missing
cp env.example .env

# Edit .env file with your API key
nano .env
```

2. Module Import Errors

```
ModuleNotFoundError: No module named 'openai'
```

Solution:

```
# Activate conda environment
conda activate agent_env

# Install dependencies
pip install -r requirements.txt
```

3. CORS Errors (Frontend Integration)

```
Access to fetch at 'http://localhost:8000' from origin 'http://localhost:3000' has been blocked by CORS policy
```

Solution:

```
# Check ALLOWED_ORIGINS in .env
echo $ALLOWED_ORIGINS

# Update .env file
ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

4. High Processing Times

Symptoms: Processing times > 30 seconds

Solutions:

- Check internet connection
- Verify OpenAI API status
- Use optimized validation mode
- Monitor token usage

5. Cost Tracking Issues

Symptoms: No cost information in logs

Solution:

```
# Enable cost tracking in .env
TRACK_API_COSTS=true
LOG_LEVEL=INFO
```

Debug Mode

Enable detailed logging:

```
# Set debug mode
export DEBUG=true
export LOG_LEVEL=DEBUG

# Restart server
python start.py
```

Performance Monitoring

Key Metrics to Monitor:

- Processing time per request
- Token usage per agent
- Cost per validation
- API response times
- Error rates

Monitoring Commands:

```
# Check server status
curl http://localhost:8000/api/system/status

# Monitor logs
tail -f logs/app.log

# Check memory usage
ps aux | grep python
```

Future Enhancements

Planned Improvements

1. Advanced Caching

- Redis-based caching for repeated validations
- File hash-based cache invalidation
- Cost optimization through cached responses

2. InHouse Knowledge Base

- Fine-tuned models for building codes

3. Scalability Improvements

- Horizontal scaling with load balancers
- Database integration for validation history
- Queue-based processing for high-volume requests

4. Advanced Reporting

- Interactive PDF reports with diagrams
- 3D visualization of design issues
- Automated correction suggestions

Performance Targets

Metric	Current	Target
Processing Time	15-20s	< 10s
Cost per Validation	\$0.02	< \$0.01
Concurrent Requests	1	10+

Useful Commands

```
# Start server
python start.py

# Check API docs
open http://localhost:8000/docs

# Test health endpoint
curl http://localhost:8000/

# Monitor logs
tail -f logs/app.log

# Check environment
conda info --envs
```

Performance Benchmarks

Current System Performance:

- **Processing Time:** 15-20 seconds average
 - **Cost per Request:** ~\$0.02
 - **Token Usage:** 1,470-1,770 tokens per validation
 - **Success Rate:** > 95%
-